SOFTWARE ENGINEERING PROJECT PROPOSAL

MCP-Powered AI Assistant

An Intelligent Personal Assistant Platform

Integrating Email, Telegram & Context-Aware Automation

Course Title: Software Engineering and Information System Design Lab

Course Code: CSE-3104

Year & Semester: 3rd Year 1st Semester

Team Name: Team Axis (Team No: 06)

Submitted To

Md. Samsuddoha
Assistant Professor
Dept. of Computer Science & Engineering
University of Barishal

Prepared By

Name	Roll
S.M. Shahriar Sourov	230102007
Mahfuj Abdullah	230102019
MD. Jubayer Hasan	230102037
MD. Shihab	230102039
MD. Azad	230102046
MD. Sumon	230102050

Submission Date: 15 May, 2026

Contents

Chapter 1

Formation of Team Axis

> Team Axis is a dedicated software engineering group formed with the primary objective of designing, developing, and delivering the **MCP-Powered AI Assistant** platform—an intelligent personal assistant system that bridges artificial intelligence with real-world communication services.

The team comprises six members bringing together a complementary mix of technical skills spanning backend development, frontend engineering, database design, AI integration, system testing, and technical documentation. Together, the team addresses every phase of the Software Development Life Cycle (SDLC) from initial requirements gathering through final deployment and maintenance.

The project is motivated by the growing complexity of modern communication workflows and the pressing need for centralized AI-driven automation. Each member's role has been deliberately assigned to leverage individual strengths while promoting knowledge sharing and cross-functional collaboration throughout the development process.

1.1 Team Composition and Responsibilities

1.1.1 Architecture & Deployment Lead — Mahfuj Abdullah (230102019)

Mahfuj Abdullah serves as the overall technical lead and system architect for the MCP-Powered AI Assistant project. His responsibilities span the full breadth of backend infrastructure, from high-level architectural decisions to hands-on deployment management.

Core Responsibilities:

- Coordinate overall project planning, sprint scheduling, and development workflow management.
- Design the complete system architecture, including backend communication flow, API gateway structure, and layered service separation.
- Lead the Model Context Protocol (MCP) integration, defining tool schemas,

communication protocols between the AI layer and backend services, and orchestration logic.

- Supervise all backend API services and ensure seamless integration with the AI model through properly defined MCP tool interfaces.
- Manage deployment pipelines on platforms such as Render and Vercel, and oversee containerization using Docker.
- Ensure structural cohesion between the AI layer, backend layer, and PostgreSQL database systems.

1.1.2 Requirement Analyst & Database Planner — MD. Azad (230102046)

MD. Azad focuses on the analytical and planning dimensions of the project, ensuring that technical solutions are grounded in well-defined and validated requirements.

Core Responsibilities:

- Conduct thorough analysis of functional and non-functional software requirements and prepare detailed system specification documents.
- Design the relational database schema, defining table structures, foreign key relationships, indexing strategies, and constraints.
- Develop Entity-Relationship (ER) diagrams, Data Flow Diagrams (DFDs), and Use Case models that serve as blueprints for implementation.
- Collaborate with the backend team on API structure planning and ensure database queries are optimized for performance.
- Review and validate requirement traceability throughout the project lifecycle.

1.1.3 Database & API Developer — MD. Jubayer Hasan (230102037)

MD. Jubayer Hasan is responsible for transforming planned database schemas and API specifications into functional, production-ready code using modern backend technologies.

Core Responsibilities:

- Develop and maintain RESTful backend APIs using the Express.js framework on Node.js runtime.

- Design and implement Prisma ORM models that map directly to PostgreSQL database tables with proper relationship definitions.
- Write efficient SQL queries and optimize database transactions to ensure low-latency data access.
- Implement authentication middleware, input validation pipelines, and error handling modules.
- Collaborate with the MCP integration layer to ensure backend tools are properly exposed and callable by the AI model.

1.1.4 Frontend Developer & Documentation Officer — MD. Shihab (230102039)

MD. Shihab bridges the gap between backend functionality and user experience, ensuring that the platform's interface is intuitive, responsive, and well-documented.

Core Responsibilities:

- Design and implement responsive user interfaces using React.js and Tailwind CSS, ensuring compatibility across desktop and mobile devices.
- Integrate the frontend chat interface with backend AI endpoints, managing state using the Context API.
- Prepare comprehensive project documentation including technical reports, user guides, API reference documents, and presentation materials.
- Maintain visual design consistency and ensure accessible, user-friendly interaction flows within the assistant interface.
- Produce and update architectural diagrams, sequence diagrams, and workflow illustrations throughout the development cycle.

1.1.5 Tester & Backend Support — MD. Sumon (230102050)

MD. Sumon plays a critical role in ensuring the platform's reliability and correctness through structured testing practices at every stage of development.

Core Responsibilities:

- Design and execute unit tests and integration tests for individual modules and cross-service communication flows.
- Identify, document, and track software bugs and defects using structured issue management practices.
- Perform comprehensive API testing using Postman, validating endpoints for correctness, edge-case handling, and performance under load.
- Verify bug fixes and regression test affected modules to ensure quality is maintained after updates.
- Monitor overall software quality metrics and provide feedback to the development team for continuous improvement.

1.1.6 Maintenance & Quality Assurance Support — S.M. Shahriar Sourov (230102007)

S.M. Shahriar Sourov ensures the long-term stability and maintainability of the platform, focusing on operational quality assurance and version management.

Core Responsibilities:

- Monitor software stability across environments and manage ongoing maintenance tasks and dependency updates.
- Support deployment workflows and coordinate version control management through GitHub, including branching strategies and code review processes.
- Assist in backend performance optimization, profiling bottlenecks, and implementing improvements.
- Ensure overall quality assurance by reviewing code, monitoring system health, and verifying deployment integrity.
- Maintain system documentation for operational runbooks and handover protocols.

1.2 Team Organizational Structure

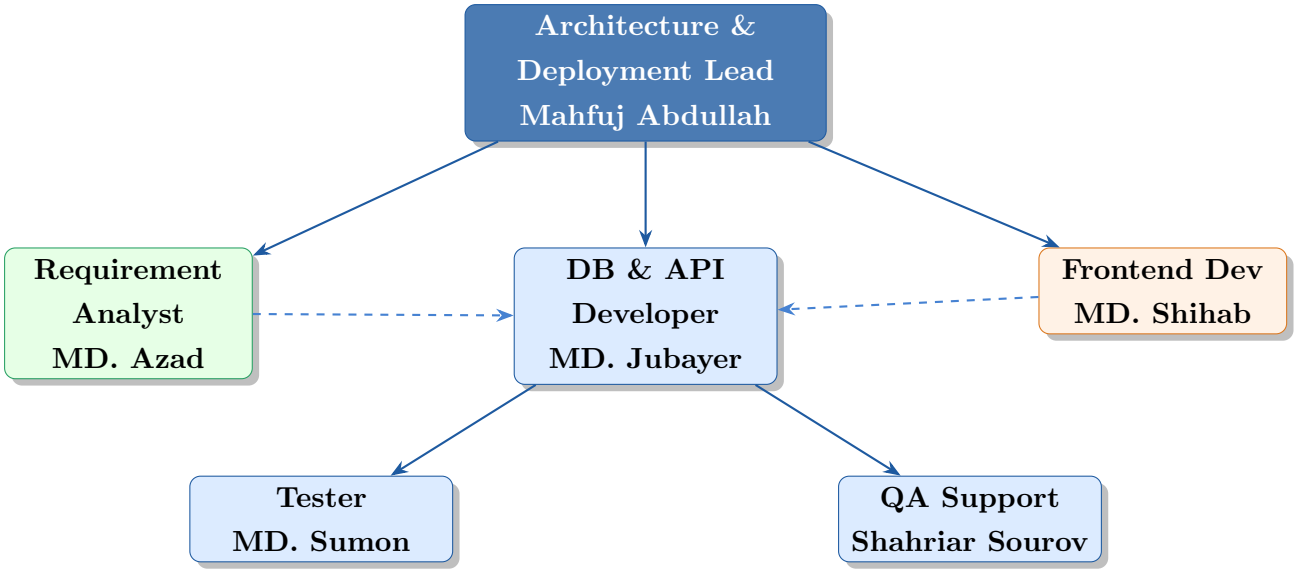

Figure 1.1: Organizational Structure of Team Axis

1.3 SWOT Analysis of Team Axis

The following SWOT analysis provides a structured evaluation of Team Axis's internal capabilities and external context as it relates to delivering the MCP-Powered AI Assistant project.

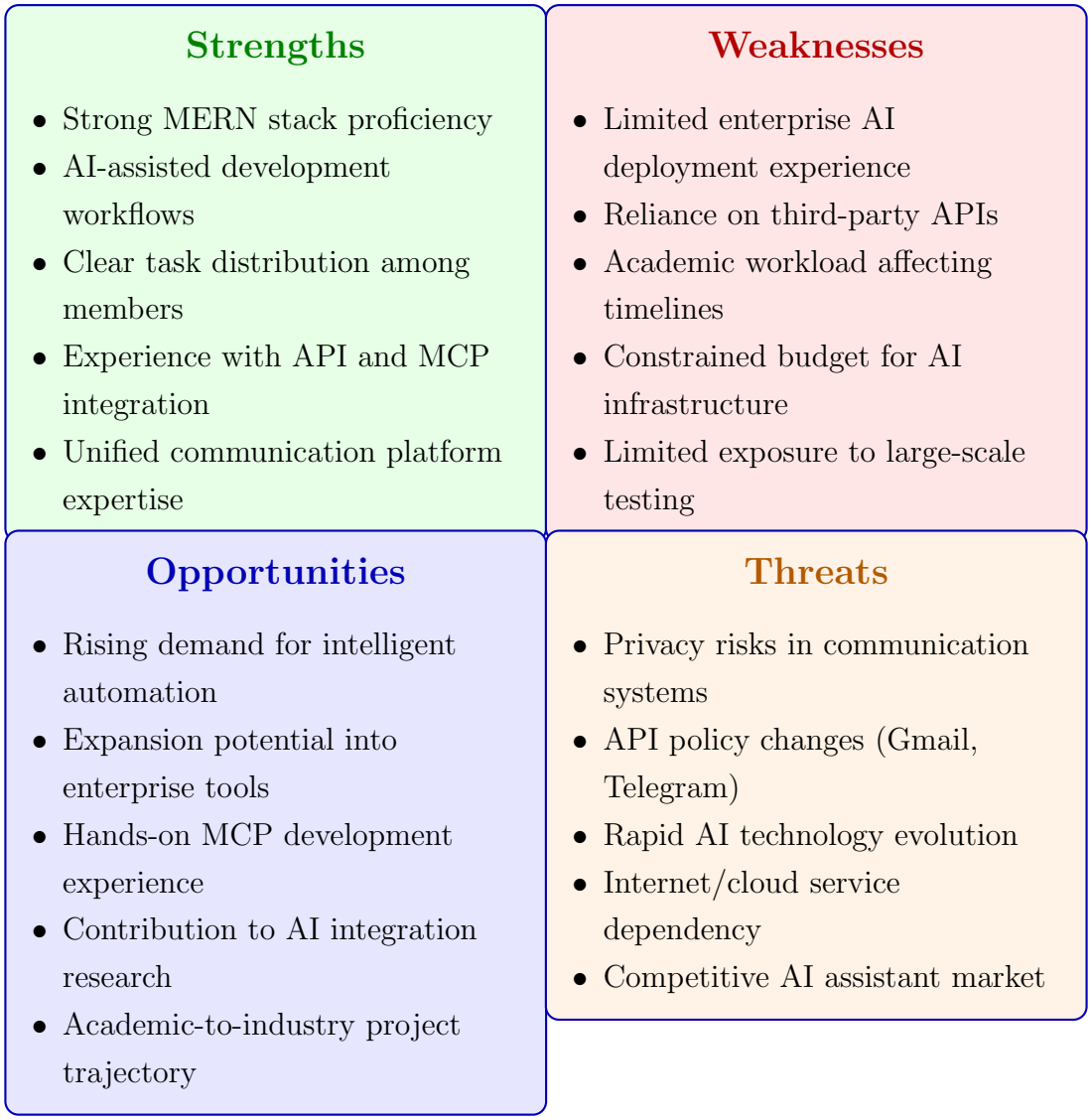

Figure 1.2: SWOT Analysis of Team Axis

Chapter 2

Overview of the MCP-Powered AI Assistant

Project Summary

The **MCP-Powered AI Assistant** is an intelligent, unified communication management platform that integrates Email automation, Telegram message handling, contact management, and AI-driven contextual assistance into a single conversational interface. It leverages the **Model Context Protocol (MCP)** to enable an AI language model to perform real-world operations directly.

Modern professionals and students interact with a growing number of digital communication services simultaneously—managing inboxes, group chats, contact lists, and scheduling notifications across separate applications. This fragmentation results in reduced productivity, communication delays, and information overload. Traditional AI chatbots, despite their conversational sophistication, remain confined to text-based responses and lack the ability to take direct action within external services.

The MCP-Powered AI Assistant addresses this fundamental gap. Rather than simply answering questions, the platform orchestrates live operations—fetching unread emails, summarizing Telegram group discussions, sending messages on behalf of users, and resolving contact identities using natural language—all through a seamless conversational interface.

2.1 Key Design Principles

- **Centralization:** All communication services accessible from one interface.
- **Modularity:** Each service (Email, Telegram, Contacts) is an independently deployable module.
- **Scalability:** Architecture supports adding new tools and services without redesign.
- **Security:** OAuth2-based authentication and encrypted token storage.
- **Extensibility:** MCP tool schema allows new capabilities to be added in-

crementally.

- **Usability:** Natural language commands replace complex UI navigation.

2.2 System Architecture Overview

The platform follows a three-layer architecture designed for clean separation of concerns:

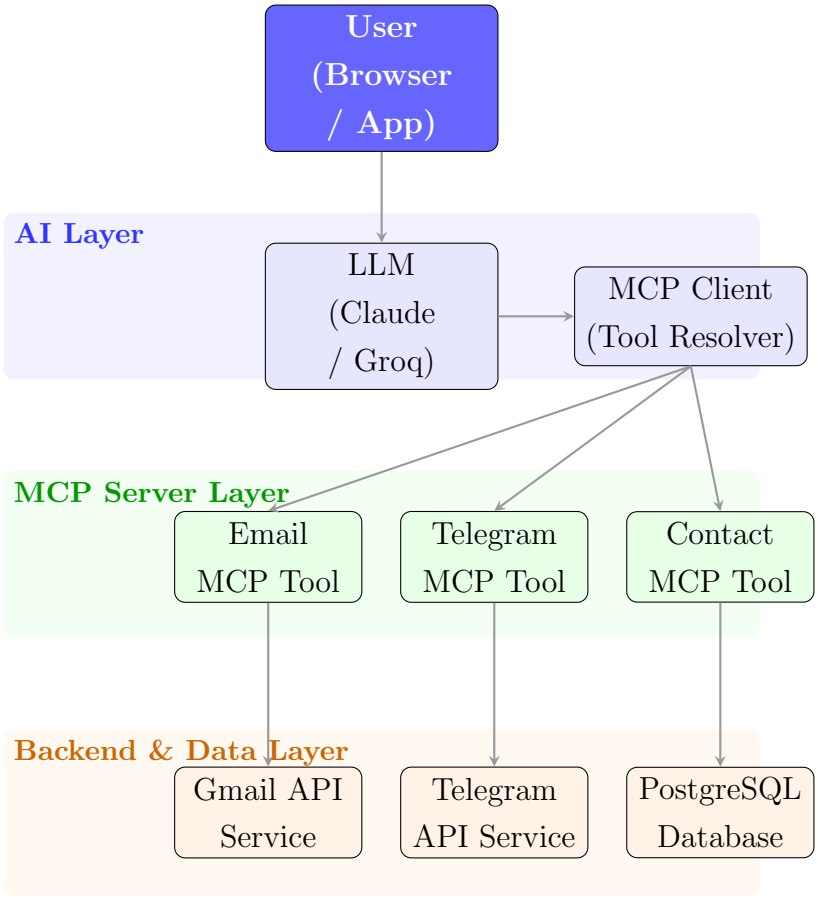

Figure 2.1: Three-Layer Architecture of the MCP-Powered AI Assistant

AI Layer Hosts the large language model (LLM) and the MCP client responsible for parsing user intent, selecting appropriate tools, and constructing tool call arguments in real time.

MCP Server Layer Contains independently deployable MCP tool servers that expose standardized tool schemas. Each tool server communicates with a specific external service via its native API.

Backend & Data Layer Provides REST API endpoints, business logic execution, authentication management, and persistent data storage through PostgreSQL via Prisma ORM.

Chapter 3

Problem Statement

3.1 Background and Context

The modern digital communication landscape is characterized by fragmentation. A typical user manages at least three to five distinct communication platforms daily—email clients, instant messaging apps, social media inboxes, and productivity tools—each operating in isolation without cross-platform awareness or intelligent assistance. This fragmentation leads to:

- **Cognitive overload:** Users must context-switch frequently, increasing mental fatigue and reducing focus.
- **Information loss:** Important messages and action items are missed or delayed due to inbox volume.
- **Repetitive manual effort:** Tasks like composing routine emails, summarizing long threads, and searching contacts are performed manually without automation.
- **Lack of centralized context:** No single assistant understands the full scope of a user's communication history and relationships across platforms.

3.2 Limitations of Existing AI Systems

Although state-of-the-art AI language models such as Claude, ChatGPT, and Gemini demonstrate remarkable natural language understanding, they remain confined to producing textual responses within isolated chat sessions. Their critical limitations include:

- **No real-world action capability:** They cannot send emails, retrieve messages, or perform operations on external services without custom integration scaffolding.
- **Session-limited memory:** Conversation context is lost between sessions; no persistent user or contact memory is maintained.

- **Platform isolation:** Each AI assistant operates independently of the user's actual communication ecosystem, making them consultative tools rather than actionable agents.
- **No intelligent summarization pipelines:** Inbox and conversation summarization is not natively integrated with live data sources.

3.3 Problem Visualization

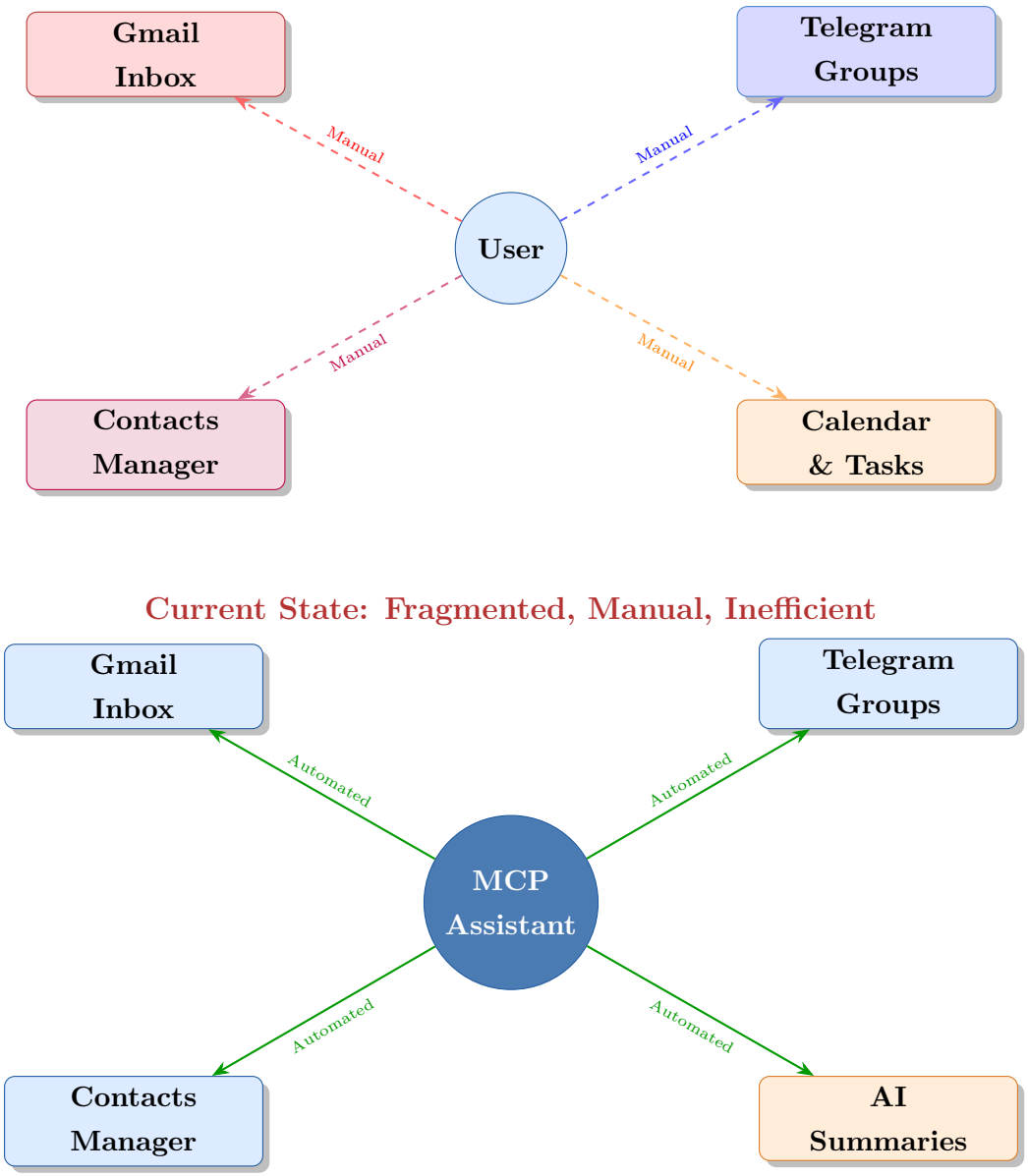

Figure 3.1: Current fragmented communication workflow (left) vs. proposed MCP-Powered AI Assistant (right)

Chapter 4

Motivation

The motivation for building the MCP-Powered AI Assistant arises at the intersection of practical necessity and technological opportunity. As communication systems grow more complex, and as AI models grow more capable, there exists a critical window to build bridging systems that unlock the practical potential of language models for everyday productivity tasks.

4.1 Practical Need

Daily communication management has become a significant overhead for professionals, students, and organizations. Surveys in productivity research consistently indicate that knowledge workers spend a disproportionate amount of their time reading, composing, and organizing communications rather than performing their primary work functions. An intelligent assistant capable of managing these functions through natural language commands would yield measurable productivity improvements.

4.2 Technological Opportunity: The MCP Ecosystem

The introduction of the **Model Context Protocol (MCP)** by Anthropic represents a significant paradigm shift in how AI systems interact with external tools. MCP provides a standardized, open protocol through which LLMs can call external tools with structured input and receive structured output—enabling AI systems to transition from passive responders to active agents.

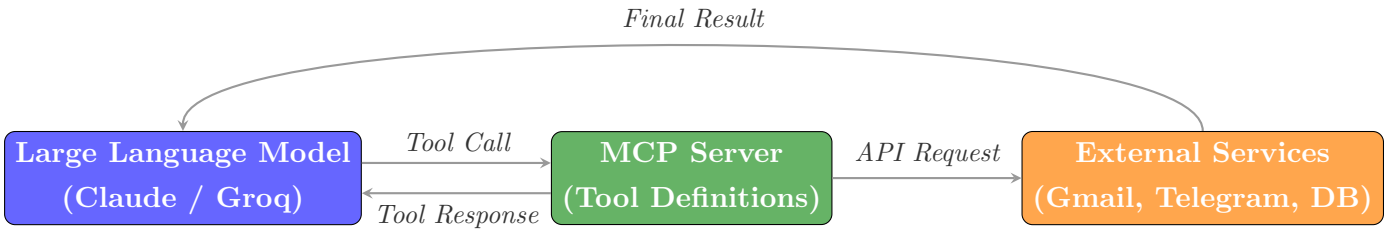

Figure 4.1: MCP Communication Flow: LLM ↔ MCP Server → External Service

4.3 Academic and Professional Value

This project provides Team Axis with hands-on experience in several advanced technical domains that are directly relevant to industry careers in software engineering and AI development:

- Practical implementation of the Model Context Protocol in a real application.
- Full-stack development using a modern MERN-adjacent technology stack.
- Integration of third-party OAuth2-authenticated APIs (Gmail, Telegram).
- Database schema design and ORM-based query management.
- Agile project management, code review workflows, and CI/CD pipelines.
- Secure software engineering practices for communication-sensitive applications.

Chapter 5

User and System Requirements

5.1 Functional Requirements

5.1.1 Email Management Module

The Email Management module integrates with Gmail through the Google OAuth2 API, allowing the AI assistant to perform a full set of email operations on behalf of authenticated users.

- **Send Email:** The system shall compose and dispatch emails using the authenticated user's Gmail account. The AI model generates contextually appropriate email content based on user instructions, including recipient address resolution through the contacts module.
- **Fetch Inbox:** The system shall retrieve email messages from the user's inbox, applying filters such as read/unread status, sender identity, subject keywords, and date ranges. Results are returned to the AI model for further processing.
- **Email Summarization:** The system shall generate concise, human-readable summaries of individual emails or batches of emails using AI-powered text summarization, reducing information overload.
- **Smart Reply Generation:** The system shall analyze incoming email context and generate contextually appropriate reply suggestions for user review and dispatch.
- **Label and Categorization:** The system shall apply intelligent labels and categories to incoming emails based on content analysis.

5.1.2 Telegram Integration Module

The Telegram Integration module uses the Telegram Bot API and MTProto client libraries to provide full message management capabilities within the assistant interface.

- **Message Retrieval:** The system shall fetch messages from specified Telegram private chats, groups, or channels, with support for pagination and time-based filtering.
- **Conversation Summarization:** The system shall produce structured AI-generated summaries of Telegram chat histories, highlighting key topics, decisions, and action items.
- **Quick Reply Assistance:** The system shall generate contextually appropriate quick-reply suggestions for Telegram conversations, enabling faster and more effective communication.
- **Message Search:** The system shall support semantic search across Telegram message history to retrieve relevant past conversations.

5.1.3 Hybrid Interaction Manager (Contact Module)

- **Contact Storage and Retrieval:** The system shall maintain a centralized contact database with fields for name, email addresses, Telegram handles, roles, organization, and relationship tags.
- **Semantic Identity Resolution:** The system shall resolve ambiguous or informal contact references (e.g., "my professor" or "the team lead") to specific contacts using contextual metadata and role-based mappings.
- **Relationship Tracking:** The system shall maintain communication history metadata per contact to support personalized communication recommendations.

5.2 Non-Functional Requirements

Category	Requirement
Performance	API response times under 2 seconds for 95% of standard tool operations.
Security	All OAuth2 tokens encrypted at rest; all API communication over HTTPS/TLS.
Availability	System uptime target of 99% during operational hours.
Scalability	Architecture supports horizontal scaling of MCP tool servers independently.
Usability	Users should be able to complete primary tasks within 3 natural language turns.
Maintainability	Codebase follows modular structure with comprehensive inline documentation.

5.3 Use Case Diagram

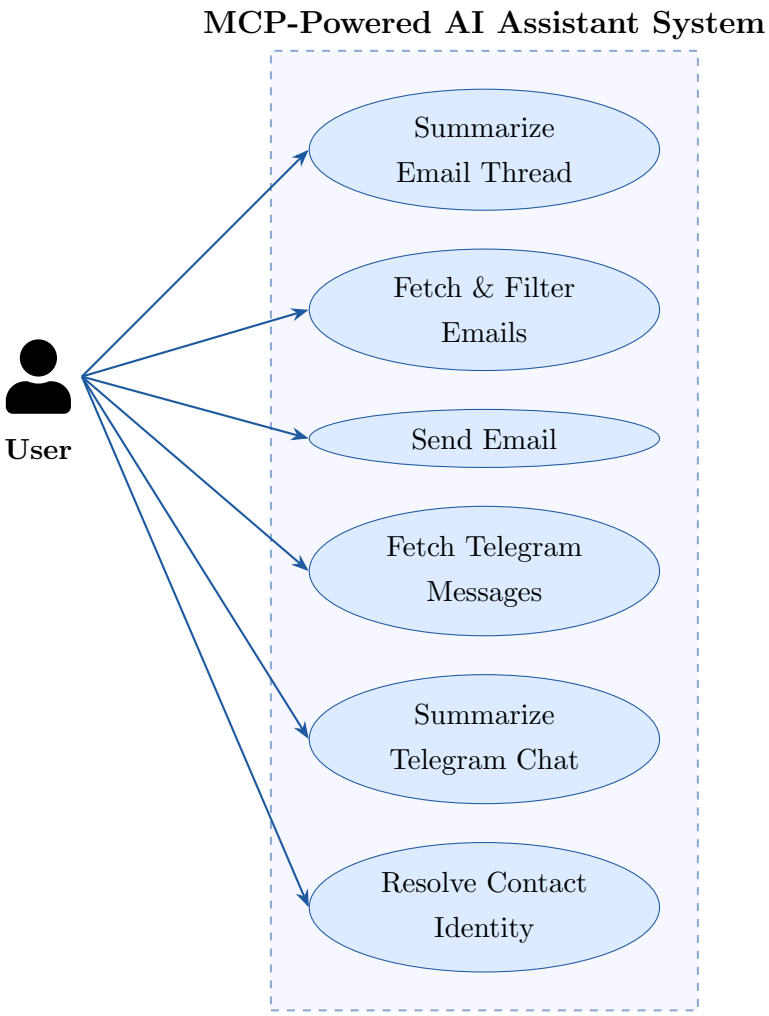

Figure 5.1: Use Case Diagram for MCP-Powered AI Assistant

Chapter 6

Development Approach

6.1 Software Development Methodology

The project adopts the **Agile Software Development** methodology, organized into weekly sprints with defined deliverables. Agile is chosen for its flexibility in accommodating evolving requirements, its emphasis on working software over comprehensive documentation, and its support for iterative refinement through testing cycles.

Each sprint follows a standard Agile cycle:

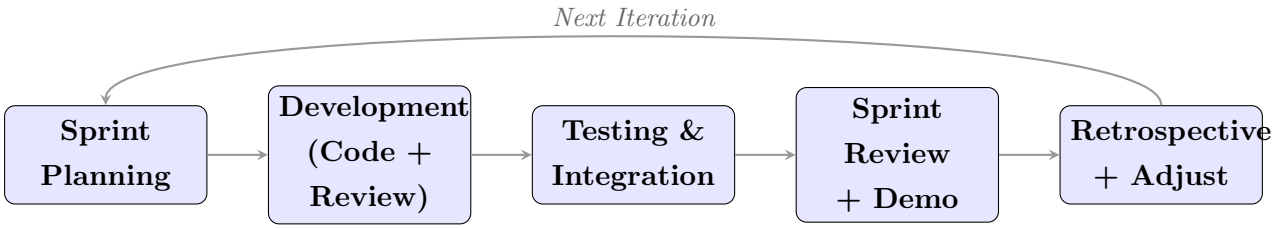

Figure 6.1: Agile Sprint Cycle for MCP-Powered AI Assistant Development

6.2 Tools and Technologies

6.2.1 Frontend Technologies

- **React.js:** Component-based SPA framework for building the conversational chat interface and dashboard views.
- **Tailwind CSS:** Utility-first CSS framework enabling rapid, consistent, and responsive UI design without custom stylesheet overhead.
- **Context API:** React's built-in state management solution used to maintain global application state including authentication status, conversation history, and user preferences.

6.2.2 Backend Technologies

- **Node.js:** Asynchronous JavaScript runtime providing high-throughput I/O performance suitable for API-heavy workloads.
- **Express.js:** Minimalist web framework for Node.js, used to define RESTful API routes, middleware pipelines, and request/response handling.
- **REST API:** Standard HTTP-based communication protocol between frontend clients, MCP tool servers, and backend services.

6.2.3 Database Technologies

- **PostgreSQL:** Robust, ACID-compliant relational database for storing user profiles, contact data, communication metadata, and session tokens.
- **Prisma ORM:** Type-safe ORM layer that generates SQL queries from schema definitions, reduces boilerplate, and provides migration management.
- **Neon Database Hosting:** Serverless PostgreSQL hosting platform offering scalable, cost-effective cloud database services with branching support.

6.2.4 AI and MCP Layer

- **Claude (Anthropic):** Primary large language model for natural language understanding, tool call generation, and contextual response synthesis.
- **Groq API:** High-speed LLM inference API providing ultra-low latency responses for time-sensitive automation tasks.
- **Ollama:** Local LLM runtime enabling offline model inference for privacy-sensitive operations.
- **Model Context Protocol (MCP):** Open standard protocol for tool integration between LLMs and external services. Used to define all tool schemas, handle tool dispatch, and process tool responses.

6.2.5 Deployment and DevOps

- **Docker:** Container platform for packaging backend services and MCP servers into portable, reproducible environments.

- **Render:** Cloud platform for deploying containerized backend services with automatic SSL, environment management, and scaling.
- **Vercel:** Edge deployment platform optimized for React.js frontend applications with built-in CI/CD and global CDN.
- **GitHub:** Version control and collaboration platform supporting branching workflows, pull request reviews, and issue tracking.

6.3 Technology Stack Overview

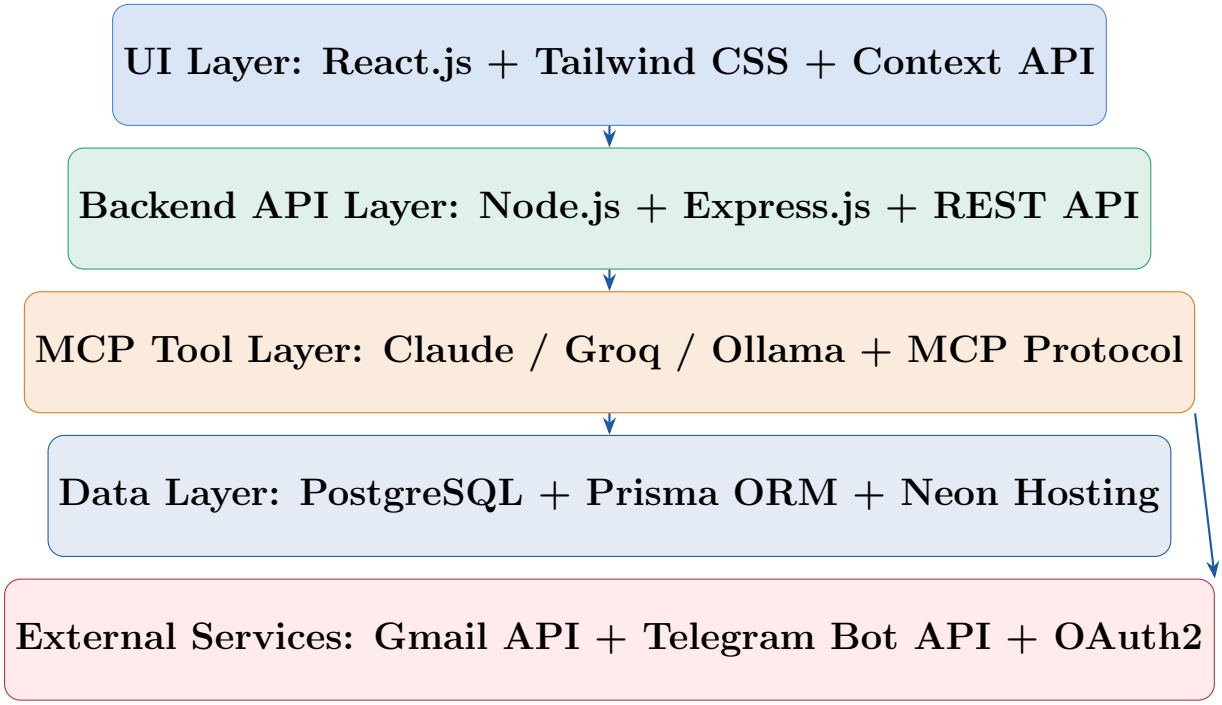

Figure 6.2: Full Technology Stack of the MCP-Powered AI Assistant

Chapter 7

Project Timeline

7.1 Development Schedule

The project spans twelve weeks, divided into structured phases aligned with the Agile sprint methodology. Each phase delivers tangible, testable components that build incrementally toward the complete platform.

Week	Phase / Module	Deliverables
1–2	Planning & Architecture	Requirements document, architecture diagrams, team role assignments, database schema draft
3–4	Backend & Database Setup	Express.js server, Prisma schema, PostgreSQL on Neon, basic REST APIs, Docker setup
5–6	Email Module	Gmail OAuth2 flow, email fetch/send APIs, MCP Email tool server, basic AI email summarization
7–8	Telegram Module	Telegram Bot API integration, message retrieval, conversation summarization, MCP Telegram tool
9	AI Orchestration	Full MCP orchestration flow, contact resolution, semantic identity mapping, quick-reply generation
10	Testing & QA	Unit tests, integration tests, Postman API test suite, bug fixes, performance profiling
11	Deployment	Docker containerization, Render backend deployment, Vercel frontend deployment, E2E testing
12	Documentation & Presentation	Final report, user guide, API documentation, project presentation slides

7.2 Gantt Chart Overview

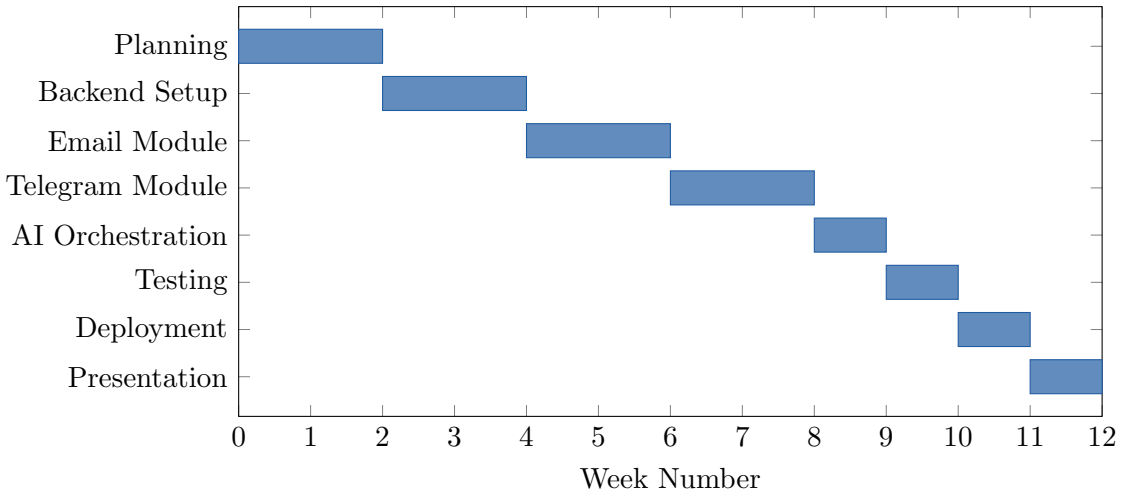

Figure 7.1: Gantt Chart Showing Module Development Phases (Weeks 1–12)

Chapter 8

Budget and Cost Estimation

8.1 Cost Breakdown

The following budget estimation covers all anticipated expenses for developing, deploying, and maintaining the MCP-Powered AI Assistant throughout the 12-week project timeline. Costs are estimated in Bangladeshi Taka (BDT).

Category	**Description**	**Estimated Cost (BDT)**
Development & Research	Team effort for backend API development, MCP integration, frontend development, and AI orchestration logic	250,000
Cloud Hosting	Neon PostgreSQL database hosting, Render backend deployment, Vercel frontend hosting	60,000
AI Services & APIs	Groq API usage credits, Claude API tokens, Gmail API (free tier), Telegram Bot API (free)	80,000
Internet & Connectivity	Team internet connectivity expenses during the development period	30,000
Documentation & Printing	Report printing, presentation materials, binding, and visual design assets	10,000
Contingency Reserve	Buffer for unexpected API cost overruns, hardware issues, or extended development needs	50,000
Total		**480,000**

8.2 Budget Distribution Chart

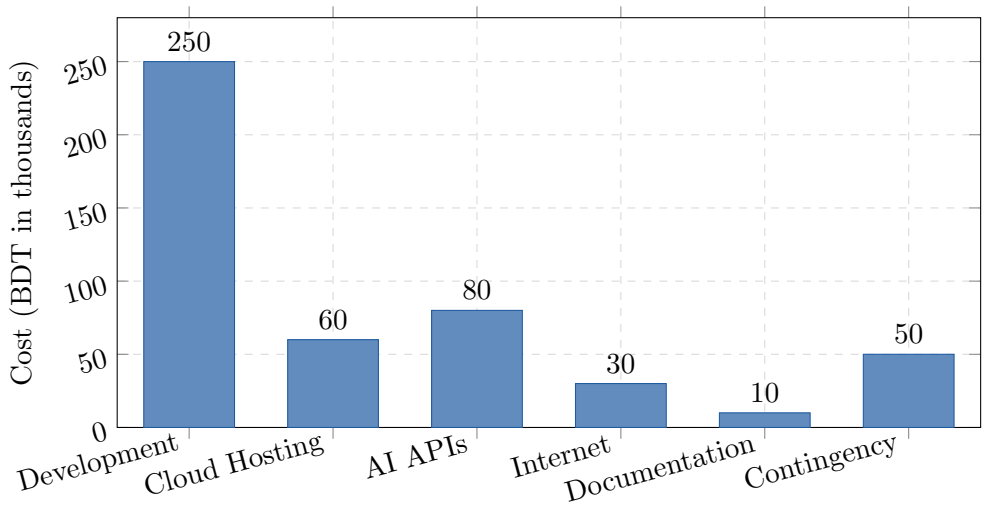

Figure 8.1: Budget Distribution by Category (BDT)

Chapter 9

Conclusion

The **MCP-Powered AI Assistant** represents a meaningful step toward the practical realization of AI-driven communication automation. By combining the conversational power of large language models with the structural capability of the Model Context Protocol, the platform transforms an AI assistant from a passive information tool into an active agent capable of performing real-world operations across multiple communication services.

9.1 Summary of Contributions

- **Unified Communication Management:** The platform consolidates Email and Telegram management into a single intelligent interface, eliminating the need to context-switch between applications.
- **MCP-Based Tool Integration:** By implementing the Model Context Protocol, the project demonstrates a scalable, standards-compliant approach to extending AI model capabilities with external service tools.
- **AI-Powered Automation:** Automated email summarization, smart reply generation, and Telegram conversation analysis significantly reduce communication overhead for end users.
- **Modular, Scalable Architecture:** The layered architecture ensures that individual modules can be maintained, upgraded, or replaced independently without disrupting the overall system.
- **Practical Engineering Experience:** The project provides Team Axis with direct, hands-on experience in building AI-integrated full-stack systems using modern industry-standard tools and methodologies.

9.2 Future Scope

The current scope of the MCP-Powered AI Assistant is intentionally focused on core communication use cases. Future iterations may extend the platform in the following directions:

- Integration with additional communication platforms such as Slack, WhatsApp Business, and Microsoft Teams.
- Development of a persistent memory layer that retains user context, preferences, and relationship history across sessions.
- Enterprise deployment with multi-user support, role-based access control, and organization-level contact management.
- Voice interface integration enabling hands-free communication management through speech-to-text and text-to-speech pipelines.
- Fine-tuning of the underlying LLM on domain-specific communication data to improve response personalization and accuracy.

Final Statement

The MCP-Powered AI Assistant project demonstrates that AI systems can be designed not merely as conversational interfaces, but as intelligent operational agents that actively participate in users' communication workflows. Through disciplined engineering, collaborative teamwork, and thoughtful integration of emerging AI protocols, Team Axis aims to deliver a platform that is both technically rigorous and genuinely impactful.

— END OF Proposal —